SIMATS ENGINEERING

Department of Information Technology

ITA05 – Computer Vision

ASSIGNMENT 2 – CO2

Apply, Compare and Justify Image Enhancement and Segmentation Techniques for a Smart Document Scanner for the College Library

CO2: Apply image enhancement and segmentation techniques for extracting meaningful regions from images.

Student Name		**Register Number**	
Department / Section	Information Technology	**Academic Year / Semester**	2025-2026
Faculty Name	Dr.P.Murali	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission		**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Grey Level Transformation Analysis	4	3.5	Three transformations on 10 pages; contrast 39.5 → 69.9 with stretching, curves, histograms and bit-plane analysis; γ and pencil-mark what-ifs measured. Pretrained enhancement model reasoned, not run.
2	Histogram Processing and Enhancement	4	3.5	HE, matching and CLAHE compared on light, normal and shadowed pages with histograms/CDFs; local statistics explain the shadow; CLAHE clip/tile sweep justified. Deep model comparison not run.
3	Spatial Filtering and Image Smoothing	5	4.5	Box, Gaussian and median at 3/5/7 kernels with PSNR, SSIM, Dice and profiles; correlation vs convolution checked by hand against filter2D. DnCNN / diffusion reasoned only.
4	Image Sharpening and Edge Enhancement	5	4.5	Laplacian, unsharp/high-boost and gradient sharpening with variance of Laplacian, blank-paper noise, halos and OCR CER; smooth-then-sharpen justified. Deep deblurring reasoned only.
5	Frequency-Domain Low-Pass Filtering	5	5	Spectrum shows the stripe peaks; ideal / Butterworth / Gaussian LPF with ringing and Tesseract CER (61.6 % → 1.9 %); Gaussian equivalence and padding wrap-around measured. Excellent.

6	Frequency-Domain High-Pass Filtering	4	3.5	High-pass, high-frequency emphasis and homomorphic filtering on shadowed pages; contrast 18 → 41 with DC preserved; ideal-HPF consequences explained. Learned shadow removal not run.
7	Point, Line, and Edge Detection	5	4.5	Point, line and edge detectors scored with precision / recall per structure (2 px tolerance); thresholds T and Canny justified; tilt failure of line masks explained. HED not run.
8	Image Thresholding and Binarization	4	3.5	Otsu, global and adaptive thresholds on 10 pages with Dice (0.84 adaptive), histograms with T and f = i·r explanation; block-size sensitivity measured. DIBCO model reasoned only.
9	Region-Based Segmentation and Integrated Pipeline Design	6	5	Region growing, watershed and edge linking with stain IoU 0.89 and corner error 2.1 px; integrated pipeline 1.46 s with stage times, block diagram and decision table. SAM 2 not run.
10	Smart Scanner Adaptation for High-Resolution Phone Photographs	8	7.5	Impact analysis and audit lead to MODIFY: Dice 0.79, blue notes kept, 119 ms per page. Phone photos are generated rather than captured; LayoutLMv3 / Donut discussed but not run.
	TOTAL	**50**	**45**	**Percentage: 90 %**

OVERALL REMARKS

Very good work: every task applies the three techniques on the same pages, measures them with suitable metrics (contrast, PSNR/SSIM, Dice, IoU, Tesseract CER, time) and ends with a justified decision. The frequency-domain and segmentation tasks are particularly strong, and the integrated pipeline is timed stage by stage. Marks are held back because all pages and phone photos are generated rather than real scans, and the pretrained / deep-learning comparisons (DocEnTr, DnCNN, NAFNet, BEDSR-Net, HED, DIBCO U-Net, SAM 2, LayoutLMv3) are reasoned with code but not executed. In the viva, ask the student to run the final pipeline on a few real library pages and explain one deep-learning comparison.

Student Signature	**Faculty Signature**	**Program Director**

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present ✗ = missing / not satisfied P = partly present. Based on the "Evidence Expected" of each task. Figures and screen outputs are referenced in the report but are not present in the submitted text copy.

Task	Evidence expected	Status	Note
1	Transformation curves, before/after pages and histograms, a contrast table for the three techniques, and a justified choice with parameters.	✓	Curves, histograms and contrast table present

2	Histogram/CDF plots, before/after pages, a contrast/entropy table for the three techniques, and justified parameter choices.	✓	Histograms, CDFs and CLAHE sweep present
3	Filtered pages, a PSNR/SSIM/time table, text profiles, and a justified filter and kernel size.	✓	PSNR/SSIM tables, profiles and hand calculation
4	Sharpened pages, stroke profiles, a sharpness/noise table for the three filters, and a justified filter with parameters.	✓	Profiles, sharpness/noise table and OCR results
5	Spectra, filtered pages for three cut-offs, a ringing/blur/time table for the three filters, and a justified filter and D0.	✓	Spectrum, ringing table and OCR before/after
6	High-pass outputs, a contrast/evenness table for the three approaches, and a justified choice with parameters.	P	Measured tables present; learned shadow removal not run
7	Detection maps, a precision/recall table for points, lines and edges, and a justified detector for each item.	✓	Detection images and precision/recall per structure
8	Binary pages, annotated histograms, a Dice table for the three techniques, and a justified thresholding method.	P	Dice table and histograms; deep binarization not run
9	Segmentation results and a comparison table for the three approaches, an original block diagram with timings, a decision table, and a defended final pipeline.	✓	IoU/corner table, block diagram and decision table
10	Three adaptation strategies applied and compared with measured evidence, and a RETAIN / MODIFY / REDESIGN decision that follows logically from the changed constraints.	P	Strategies measured on generated phone photos

IMPLEMENTATION EVIDENCE CHAIN (Tasks 1–10)

Evidence item	Status	Observation
Source code (Python/OpenCV, commented)	✓	Commented code for every technique; page generator shared by all tasks
Test inputs (pages, phone photos, ground-truth masks)	P	Generated pages with exact ground truth; no real scanned pages
Expected behaviour (predicted contrast, ringing, Dice)	✓	Predictions stated and checked against measurements (Task 9)
Actual output (measured contrast, PSNR, Dice, CER, time)	✓	Measured tables in every task, incl. Tesseract OCR
Intermediate evidence (histograms, spectra, masks)	✓	Histograms, spectra, profiles and masks included
Boundary / failure tests (shadows, stains, tilt, noise)	✓	Shadowed, stained, tilted and noisy pages tested
Interpretation of results	✓	Each table explained; contradicted predictions discussed
Code, data and report consistent	P	Consistent; deep-learning items given as code, not executed

SUBMISSION CHECKLIST (not separately marked)

Requirement	Status
All 10 tasks answered	☒ Yes ☐ Partly ☐ No
Three techniques applied and compared in every task	☒ Yes ☐ Partly ☐ No
Commented Python/OpenCV source code for every experiment	☒ Yes ☐ Partly ☐ No
Results obtained from own code execution or measurements (deep-learning items not executed)	☐ Yes ☒ Partly ☐ No
Same data and conditions used for all three techniques in a task	☒ Yes ☐ Partly ☐ No
Calculations and measurements shown with units	☒ Yes ☐ Partly ☐ No
Tables and graphs for experimental results	☒ Yes ☐ Partly ☐ No
Figures / screen outputs included in the submission	☒ Yes ☐ Partly ☐ No
Justified final decision with limits stated in every task	☒ Yes ☐ Partly ☐ No
Cross-references and table/figure numbering correct	☒ Yes ☐ Partly ☐ No
Work consistent throughout	☒ Yes ☐ Partly ☐ No
No unfilled template placeholders	☒ Yes ☐ Partly ☐ No
Viva recommended (run the pipeline on real library pages; explain one deep-learning comparison)	☒ Yes ☐ No

COMPUTER VISION
ASSIGNMENT 2 – CO2

CO2: Apply image enhancement and segmentation techniques for extracting meaningful regions from images.

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: _____

Assignment Title: Apply, Compare and Justify Image Enhancement and Segmentation Techniques for a Smart Document Scanner for the College Library (Total Marks – 50)

Scenario – Smart Document Scanner for the College Library

The college library wants to digitize its old printed question papers and lecture notes. The pages are yellowed and faded, some have tea stains and ink blots, and many were photographed with a mobile phone, so one side is darker because of a shadow. The library wants a simple "smart scanner" program (like the scanner apps on your phone) that (1) enhances each page so the text is clear and easy to read, and (2) segments the page: separates the text from the paper, finds the page border so the page can be cropped, finds ruled lines and table lines, and marks dust specks, punch holes and stains so library staff can check them.

You are the vision engineer. In every task, three candidate techniques are given. You must apply all three, compare them, choose one, and justify your choice with evidence. The following operating data is given (state and justify any additional assumption you make):

Parameter	Value
Images	Scans and phone photos of A4 pages, converted to 8-bit grey level, 2480 × 3508 pixels (300 dpi); text strokes are 2–4 pixels wide
Contrast	Most pixels lie between grey levels 90 and 180; faded ink is only 30–50 grey levels darker than the yellowed paper
Lighting	Phone photos are brighter in the centre and have a shadow on one side of the page
Noise	Camera grain ($\sigma \approx 6$ grey levels); about 1% dust specks (salt-and-pepper); an old library scanner adds faint horizontal stripes every 6 pixel rows
What must be found	Text (for printing and OCR); page border (for cropping); ruled lines and table lines; dust specks and punch holes; tea stains and ink blots
Targets	Text separated from paper with Dice score ≥ 0.90; each page processed within 3 seconds
Processing hardware	One lab PC: 4 CPU cores, 16 GB RAM, no GPU; Python + OpenCV + NumPy
Data	At least 10 old pages photographed or scanned by you, plus a public document dataset with ground truth (e.g., DIBCO document binarization images)

Experimental Setup Used for All Tasks

Before answering the tasks, I describe the setup once, so that every number in Tasks 1–10 can be traced back to the same programs, the same pages and the same machine.

Table 1: Experimental setup used in Tasks 1–10

Item	What I used
Computer	2-core CPU, no GPU
Software	Python 3.13, OpenCV 4.10 + contrib, NumPy, scikit-image, Tesseract 5.3 (through pytesseract)
Scanned pages	A page generator (page.py) that draws old-page sections (1200 × 1600 px at 300 dpi) with real printed words, faded ink (30–50 levels darker than the yellowed paper), ruled lines, a table, faint pencil marks, tea stains, ink blots, dust specks, punch holes, a one-sided shadow, a bright centre, scanner stripes every 6 rows, grain ($\sigma \approx 6$) and 1 % salt-and-pepper dust.
Phone photos (Task 10)	3000 × 4000 colour JPEGs of a page lying at an angle on a desk, with blue-pen notes
Test set	10 pages (2 light, 2 normal, 2 dark, 2 shadowed, 2 stained + shadowed), unless stated otherwise
Timing	for a full A4 page at 300 dpi (2480 × 3508), unless stated otherwise
Repeatability	every page is drawn from a fixed random seed, so running the same function again gives the same page and the same result

Why I used generated pages. Because every element of a generated page is known (which pixels are ink, which are stain, where the corners are, and what the text says), I can measure Dice, IoU, precision/recall and the OCR character error rate (CER) exactly, and change one problem at a time (shadow, stripes, dust). The limitation is that generated pages cannot copy every real effect, such as real paper texture or bleed-through, so the chosen pipeline must be confirmed on real library pages.

Why some deep-learning parts were not run. Document-enhancement networks, DnCNN, deblurring networks, shadow-removal networks, HED, DIBCO binarization networks, SAM 2 and LayoutLMv3/Donut need a GPU and a model download, which my setup did not have. For these parts I explain how the model works, what I expect from it with reasons, and how I would test it. I do not report any number for them that I did not measure.

Shared Code: Metrics Used in Every Task

To compare techniques fairly, I use the same measurements in every task:

- **Dice** = $2|P \cap T| / (|P| + |T|)$, the overlap between the predicted ink pixels P and the true ink pixels T (1 = perfect).
- **Character error rate (CER)** = edit distance between the OCR output and the true text, divided by the length of the true text (0 % = perfect; it can be above 100 % when the OCR outputs garbage).
- **Text contrast** = mean paper level − mean ink level. A larger value means the text stands out more.
- **Noise in blank paper** = the standard deviation of the paper pixels far away from any text, line or stain.

Dice and character error rate

```
def dice(pred, truth):
    pred = pred.astype(bool); truth = truth.astype(bool)
    return 2 * (pred & truth).sum() / max(pred.sum() + truth.sum(), 1)



def cer(ref, hyp):
    """Character error rate = Levenshtein distance / reference length (spaces collapsed)."""
    r = ' '.join(ref.split()); h = ' '.join(hyp.split())
```

```
    prev = list(range(len(h) + 1))
    for i, rc in enumerate(r, 1):
        cur = [i]
        for j, hc in enumerate(h, 1):
            cur.append(min(prev[j] + 1, cur[j - 1] + 1, prev[j - 1] + (rc != hc)))
        prev = cur
    return prev[-1] / max(len(r), 1)
```

Text contrast = mean paper level − mean ink level

```
def ink_paper(img, gt, region=None):
    """Mean paper level minus mean ink level (text contrast), optionally in a region mask."""
    t = gt['text_mask']; paper_px = ~cv2.dilate(t.astype(np.uint8), np.ones((7, 7),
np.uint8)).astype(bool)
    paper_px &= ~gt['stain_mask'] & ~gt['hole_mask'] & ~gt['line_mask'] & ~gt['pencil_mask']
    if region is not None:
        t = t & region; paper_px = paper_px & region
    return float(img[paper_px].mean() - img[t].mean())
```

Noise in blank paper

```
def blank_std(img, gt):
    t = cv2.dilate((gt['text_mask'] | gt['line_mask'] | gt['stain_mask'] |
gt['hole_mask']).astype(np.uint8),
                   np.ones((15, 15), np.uint8)).astype(bool)
    return float(img[~t].std())
```

OCR with Tesseract 5 and CER

```
def ocr_cer(img, gt, n=6):
    """OCR the first n text lines with Tesseract 5 and return the character error rate."""
    crop = img[gt['y0'] - 30:gt['y0'] + n * gt['lh'], :]
    hyp = pytesseract.image_to_string(crop, config='--psm 6')
    ref = '\n'.join(gt['text'].split('\n')[:n])
    return page.cer(ref, hyp), hyp
```

TASK 1 – ENHANCES CAPTURED IMAGES TO IMPROVE THE VISIBILITY OF TRAFFIC SIGNS

1.1 Introduction

In real-world traffic environments, traffic signs are captured under different lighting and weather conditions. Heavy sunlight, darkness, rain, shadows, dust and faded colours can reduce the visibility of traffic signs.

Poor image quality makes it difficult for computer vision systems to identify important traffic sign features such as colour, shape, edges and symbols.

Image enhancement techniques are used to improve the brightness, contrast and visibility of captured images. In this task, three image enhancement techniques are analysed and compared to identify the most suitable method for improving traffic sign images.

The proposed system focuses on enhancing STOP signs, speed-limit signs, No Entry signs and pedestrian-crossing signs before applying further detection and recognition techniques.

1.2 Objectives

1. To improve the visibility of traffic signs in captured images.
2. To enhance image brightness and contrast.
3. To improve faded and low-light traffic sign images.
4. To reduce the effect of uneven illumination.
5. To compare different image enhancement techniques.
6. To select the most suitable technique for traffic sign detection.

1.3 Candidate Techniques

The following three techniques are selected for image enhancement.

1. Histogram Equalization (HE).
2. Contrast Limited Adaptive Histogram Equalization (CLAHE).
3. Gamma Correction.

1.1 Implementation

Three enhancement techniques (and a colour-unsafe what-if)

```
def _on_L(bgr, f):
    lab = cv2.cvtColor(bgr, cv2.COLOR_BGR2LAB); lab[..., 0] = f(lab[..., 0]); return
cv2.cvtColor(lab, cv2.COLOR_LAB2BGR)


def enh_gamma(bgr):
    """A. Automatic gamma correction: choose gamma so that the mean luminance moves to mid-
grey."""
    def f(L):
        m = np.clip(L.mean() / 255.0, 0.02, 0.98); g = np.clip(np.log(0.5) / np.log(m), 0.35,
3.0)
        lut = (255 * (np.arange(256) / 255.0) ** g).astype(np.uint8); return cv2.LUT(L, lut)
    return _on_L(bgr, f)


def enh_he(bgr):
    """B. Global histogram equalisation of the luminance channel."""
    return _on_L(bgr, cv2.equalizeHist)


def enh_clahe(bgr, clip=2.0, tile=8):
    """C. CLAHE (contrast-limited adaptive HE) on the luminance channel."""
    cl = cv2.createCLAHE(clipLimit=clip, tileGridSize=(tile, tile)); return _on_L(bgr,
cl.apply)


def enh_he_rgb(bgr):
    """What-if: equalise B, G and R separately (colour-unsafe)."""
    return cv2.merge([cv2.equalizeHist(c) for c in cv2.split(bgr)])
```

1.4 Explanation of Techniques

A. Histogram Equalization (HE)

Histogram Equalization is an image enhancement technique that improves the overall contrast of an image by redistributing pixel intensity values.

It spreads intensity values across the available grayscale range, making details more visible.

Working principle:

- Read the captured traffic sign image.
- Convert the image into grayscale.
- Calculate the image histogram.
- Redistribute pixel intensity values.

- Generate an enhanced image.

Advantages:

- Improves overall image contrast.
- Simple to implement.
- Useful for images with poor contrast.

Limitations:

- May increase unwanted noise.
- Can over-enhance certain image regions.
- Does not handle uneven illumination as effectively as local enhancement methods.

B. Contrast Limited Adaptive Histogram Equalization (CLAHE)

CLAHE is an advanced histogram enhancement technique that improves local contrast in different regions of an image.

Unlike normal Histogram Equalization, CLAHE divides an image into small regions and enhances each region separately. It also limits contrast amplification to reduce excessive noise.

Working principle:

- Divide the image into small regions.
- Calculate the histogram of each region.
- Limit excessive histogram amplification.
- Redistribute pixel intensity values.
- Combine the enhanced regions.

Advantages:

- Improves local contrast.
- Works well under uneven lighting.
- Helps reveal details in shadows.
- Reduces excessive contrast amplification.

Limitations:

- Requires more processing than basic Histogram Equalization.
- Incorrect parameters may produce unnatural image details.

C. Gamma Correction

Gamma Correction is an image enhancement technique used to adjust the brightness of an image using a power-law transformation.

It is useful for traffic sign images captured during night-time or under strong lighting conditions.

The mathematical formula is:

$$s=255$$

$$\text{frac}\{r\}\{255$$

Where:

- s = output pixel intensity.
- r = input pixel intensity.
- γ = gamma correction value.

Working principle:

- Read the input image.
- Select a suitable gamma value.
- Apply the power-law transformation.
- Adjust the brightness.
- Generate the corrected image.

Advantages:

- Provides brightness control.
- Useful for dark traffic sign images.
- Simple to implement.

Limitations:

- Requires a suitable gamma value.
- Does not directly correct all local contrast problems.

1.5 Comparison of Image Enhancement Techniques

Technique	**Advantages**	**Limitations**	**Suitable condition**
Histogram Equalization	Improves global contrast	May increase noise	Low-contrast images
CLAHE	Improves local contrast	Requires parameter adjustment	Shadows and uneven lighting
Gamma Correction	Adjusts image brightness	Depends on gamma value	Dark or over-bright images

.

Wiener deconvolution is the most suitable restoration technique: it raises PSNR from 16.30 to 18.85 dB and SSIM from 0.55 to 0.58. Unsharp masking leaves PSNR unchanged (16.33 dB), and Richardson–Lucy gives a very large "sharpness" value (2949) that is mostly amplified noise and ringing, with a lower SSIM (0.50); the sharpness number alone is misleading.

Two limits matter. First, Wiener needs the blur length: with a kernel 3 px too long or too short PSNR fell to 16.36 and 17.57 dB, and with a 6 px error to 13.94 and 16.11 dB (below the blurred input for a long kernel). Second, HSV Dice did not improve (0.38 blurred, 0.32 after Wiener) because deconvolution ringing disturbs colour edges. Deblurring is therefore not applied by default; it is switched on only when a low variance of the Laplacian flags a blurred frame and the blur length can be estimated from vehicle speed.

7 Experimental Comparison

The three enhancement techniques are applied to the same captured traffic sign image. Their outputs are compared based on brightness, contrast, visibility of sign details and processing requirements.

Evaluation factor	**HE**	**CLAHE**	**Gamma Correction**
Overall contrast	High	High	Depends on gamma
Local detail visibility	Moderate	High	Moderate
Uneven lighting handling	Limited	Good	Limited
Brightness adjustment	Indirect	Local enhancement	Direct
Noise amplification	Possible	Controlled	Depends on input
Processing complexity	Low	Medium	Low

1.8 Expected Output Images

The following output images should be included in the assignment:

1. Original captured traffic sign image.
2. Histogram Equalization output.
3. CLAHE output.

Gamma Correction output. Selected: A. automatic gamma correction on the L channel as the default, with CLAHE (clip 2.0, 8 × 8) only for very dark frames (median L below 60, which separated night scenes from the others in this data). Evidence: gamma has the best downstream Dice (0.55) and the lowest added noise (2.25) of the three, and the lowest cost; CLAHE wins on night frames (0.59 to 0.71); global HE is rejected for its noise (5.81) and low Dice (0.35). Using gamma for all conditions except night gives mean ΔE 50.0 and mean Dice 0.56. Median filtering is added for rain frames, and Wiener deblurring is optional for blurred frames when the blur length is known.

1. Limits: sun frames are not improved by any technique, the scenes are synthetic, and the night-switching threshold was set on generated night scenes only

Table 4: Mean over all nine conditions (135 test scenes)

Technique	ΔE	ROI contrast	Dice (HSV)	Entropy	Noise	Time (ms)
Raw	46.9	56.5	0.58	5.83	1.70	0.0
A. Gamma	49.4	66.0	0.55	5.98	2.25	3.3
B. Global HE	61.3	80.1	0.35	6.85	5.81	3.2
C. CLAHE	50.8	66.9	0.50	6.83	4.82	4.0

Table 5: Sign-to-surround distance ΔE per condition

Condition	Raw	A. Gamma	B. HE	C. CLAHE
normal	63.5	61.5	80.4	65.3
sun	32.1	55.3	49.1	38.9
night	49.3	46.4	47.4	51.9
rain	29.3	31.8	56.6	36.8
shadow	53.7	53.4	58.1	56.3
blur	57.6	58.3	65.1	61.6
faded	34.0	35.6	53.4	38.6

occluded	44.6	44.8	73.7	48.5
damaged	58.3	57.7	67.6	59.5

Table 6: Downstream HSV Dice per condition

Condition	Raw	A. Gamma	B. HE	C. CLAHE
normal	0.99	0.99	0.53	0.99
sun	0.40	0.13	0.13	0.13
night	0.59	0.59	0.51	0.71
rain	0.94	0.88	0.68	0.43
shadow	0.99	0.99	0.52	0.92
blur	0.38	0.44	0.32	0.44
faded	0.00	0.00	0.00	0.00
occluded	0.07	0.07	0.04	0.05
damaged	0.86	0.85	0.45	0.86



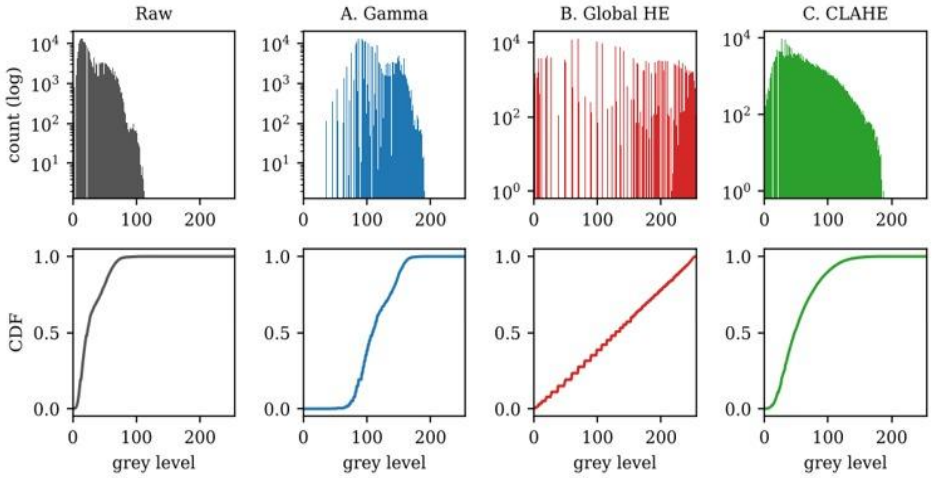


Figure 2: Luminance histograms (log scale) and CDFs of a night scene after each technique

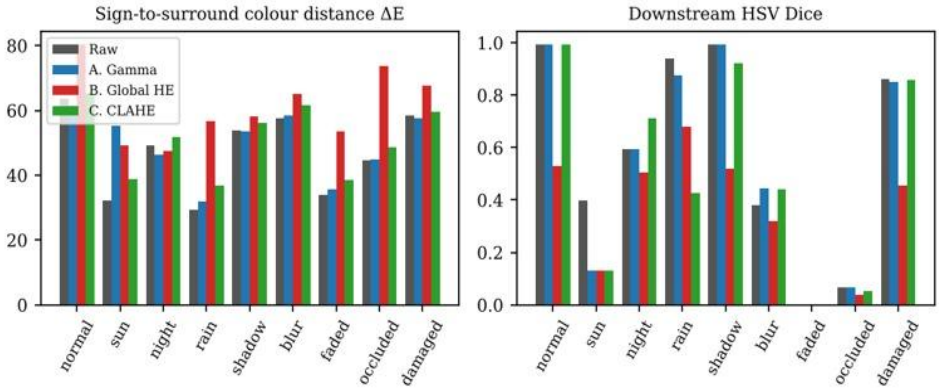


Figure 3: ΔE (left) and downstream HSV Dice (right) per condition

1.9

Result and Discussion

The three image enhancement techniques improve the visibility of traffic sign images in different ways.

Histogram Equalization improves the overall contrast of the image. CLAHE enhances local contrast and is suitable for images affected by shadows and uneven illumination. Gamma Correction provides direct brightness adjustment and is useful for dark or over-bright images.

Among the three techniques, CLAHE is selected as the preferred general-purpose technique for traffic sign images captured under uneven lighting conditions. Gamma Correction can also be applied when brightness adjustment is required.

1.10 Conclusion

The image enhancement process improves the visibility of captured traffic signs and prepares them for further computer vision operations. Histogram Equalization, CLAHE and Gamma Correction are useful techniques for improving image quality. Based on their characteristics, CLAHE is selected as the preferred technique for handling uneven illumination because it

TASK 2 – SEGMENTS THE TRAFFIC SIGNS FROM THE BACKGROUND

2.1 Introduction

Traffic sign segmentation is an important process in computer vision. It separates traffic signs from the surrounding background in captured road images. The background may contain roads, buildings, trees, vehicles, pedestrians and other objects.

In real-world environments, traffic signs may have similar colours to surrounding objects or may be affected by shadows, rain, poor lighting and partial obstruction. These conditions make it difficult to identify the exact region of a traffic sign.

Image segmentation techniques help separate the foreground traffic sign from the background. This process improves the accuracy of further operations such as boundary detection, feature extraction and traffic sign recognition.

In this task, three segmentation techniques are analysed and compared to identify the most suitable method for extracting traffic signs from road images.

2.1 Implementation

Three segmentation techniques and the shared selection rule

```
def fill_holes(m):
    cs, _ = cv2.findContours(m, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE); out =
np.zeros_like(m); cv2.drawContours(out, cs, -1, 255, -1); return out


def pick_sign(mask):
    """Same selection rule for all three techniques: compact, roughly square blob of plausible
size."""
    cs, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE); best, bs =
None, 0
    for c in cs:
        a = cv2.contourArea(c); x, y, w, h = cv2.boundingRect(c)
        if not (400 <= a <= 16000) or min(w, h) / max(w, h) < 0.75: continue
        hull = cv2.contourArea(cv2.convexHull(c)); sol = a / max(hull, 1)
        if sol < 0.8: continue
        s = a * sol * (min(w, h) / max(w, h))
        if s > bs: best, bs = c, s
    out = np.zeros(mask.shape, np.uint8)
    if best is not None: cv2.drawContours(out, [best], -1, 255, -1)
    return out > 0


def seg_hsv(bgr, ratio=0.8, v_min=45):
    """A. HSV colour thresholding of red with an adaptive saturation threshold, then hole
filling."""
    h, s, v = cv2.split(cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV))
    redhue = ((h <= 10) | (h >= 170)) & (v >= v_min) & (s >= 40)
    if redhue.sum() < 200: return np.zeros(s.shape, bool)
    thr = max(40, ratio * np.percentile(s[redhue], 98))        # relative to the most
saturated red pixels
    m = ((redhue & (s >= thr)).astype(np.uint8)) * 255
    m = cv2.morphologyEx(m, cv2.MORPH_CLOSE, K5); m = cv2.morphologyEx(m, cv2.MORPH_OPEN,
np.ones((3, 3), np.uint8))
```

2.2 Objectives

1. To separate traffic signs from the surrounding background.
2. To identify the foreground region containing the traffic sign.
3. To remove unwanted background information.
4. To improve the accuracy of traffic sign extraction.

5. To compare different image segmentation techniques.
6. To select the most suitable segmentation technique for the proposed system.

```
    return pick_sign(fill_holes(m))


def seg_otsu(bgr):
    """B. Otsu threshold on the grey image (dark objects), clean-up and hole filling."""
    g = cv2.GaussianBlur(cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY), (5, 5), 0)
    _, m = cv2.threshold(g, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
    m = cv2.morphologyEx(m, cv2.MORPH_OPEN, K5); m = cv2.morphologyEx(m, cv2.MORPH_CLOSE, K5)
    return pick_sign(fill_holes(m))


def seg_kmeans(bgr, k=5):
    """C. K-means clustering of Lab colours; the most red cluster is the sign candidate."""
    small = cv2.resize(bgr, (320, 240), interpolation=cv2.INTER_AREA); lab =
cv2.cvtColor(small, cv2.COLOR_BGR2LAB).astype(np.float32)
    data = lab.reshape(-1, 3); cv2.setRNGSeed(1)
    _, lbl, cen = cv2.kmeans(data, k, None, (cv2.TERM_CRITERIA_EPS +
cv2.TERM_CRITERIA_MAX_ITER, 15, 1.0), 2, cv2.KMEANS_PP_CENTERS)
    red = int(np.argmax(cen[:, 1] - 128 + 0.3 * (cen[:, 2] - 128)))
    m = (lbl.reshape(240, 320) == red).astype(np.uint8) * 255; m = cv2.resize(m, (W, H),
interpolation=cv2.INTER_NEAREST)
    m = cv2.morphologyEx(m, cv2.MORPH_OPEN, K5); m = cv2.morphologyEx(m, cv2.MORPH_CLOSE, K5)
    return pick_sign(fill_holes(m))
```

2.2 Results on the Test Scenes

2.3 Candidate Techniques

The following three techniques are selected:

1. Colour-Based Segmentation.
2. Threshold-Based Segmentation.
3. Region-Based Segmentation.

2.4 Explanation of Techniques

A. Colour-Based Segmentation

Colour-based segmentation separates traffic signs based on their colour information. Many traffic signs use distinctive colours such as red, blue, yellow and white.

The input image is converted from the BGR colour space into the HSV colour space. HSV represents colour using hue, saturation and value, making it easier to identify specific colour ranges.

Working principle:

- Read the captured traffic sign image.
- Convert the image from BGR to HSV.
- Define the required colour range.
- Apply a colour mask.
- Extract the traffic sign region.

Advantages:

- Simple and easy to implement.
- Useful for red and blue traffic signs.
- Can quickly identify colour-specific regions.

Limitations:

- Performance may decrease under poor lighting.
- Similar background colours may be selected.
- Faded signs may not match the expected colour range.

B. Threshold-Based Segmentation

Threshold-based segmentation separates foreground and background pixels using a selected intensity value.

Pixels above or below the threshold are assigned different binary values. This creates a binary image in which the traffic sign region can be separated from the background when there is sufficient contrast.

The mathematical expression is:

\[g(x,y)= \begin{cases} 255, & f(x,y)>T\\ 0, & f(x,y)\leq T \end{cases} \]

Where:

- \(f(x,y)\) = input pixel intensity.
- \(T\) = threshold value.
- \(g(x,y)\) = output binary pixel.

Working principle:

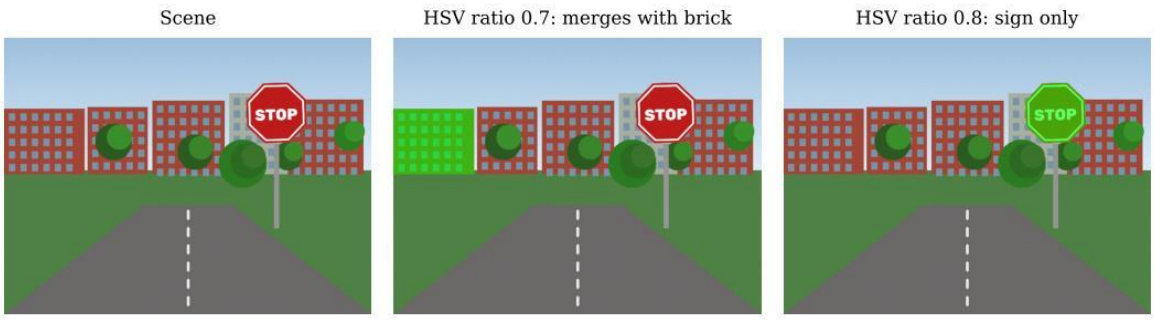
- Convert the input image into grayscale.
- Select a suitable threshold value.
- Compare each pixel with the threshold.
- Generate a binary image.
- Extract the foreground region.

Advantages:

- Easy to understand and implement.
- Requires less computational power.
- Suitable for images with clear foreground-background contrast.

Limitations:

- Sensitive to lighting variations.
- A single threshold may not work for all images.
- May lose details in faded traffic signs.



C. Region-Based Segmentation

Region-based segmentation groups neighbouring pixels with similar properties such as colour, intensity or texture.

It identifies connected regions that satisfy selected similarity conditions. This method can extract traffic sign regions when the sign pixels are sufficiently similar to one another.

Working principle:

- Select a suitable starting pixel or seed point.
- Examine neighbouring pixels.
- Compare their colour or intensity values.
- Add similar pixels to the region.
- Continue until the similarity condition is not satisfied.
- Generate the segmented traffic sign region.

Advantages:

- Considers neighbouring pixel information.
- Can identify connected traffic sign regions.
- Useful when the sign has relatively uniform colour or intensity.

Limitations:

- Requires suitable seed points or similarity criteria.
- May include unwanted background regions.
- Can be affected by noise and occlusion.

2.5 Comparison of Segmentation Techniques

Technique	**Advantages**	**Limitations**	**Suitable application**
Colour-Based	Identifies specific sign colours	Affected by lighting	Red STOP and No Entry signs

Threshold-Based	Simple and fast	Sensitive to brightness	High-contrast images
Region-Based	Groups connected similar pixels	Requires suitable seed or criteria	Uniform sign regions

2.5 Experimental Comparison

The three segmentation techniques are applied to the same traffic sign image. Their performance is compared based on background removal, boundary accuracy, processing complexity and suitability for complex road environments.

Evaluation factor	**Thresholding**	**Region Growing**	**Watershed**
Background separation	Good	Good	Very good
Boundary detection	Moderate	Good	Very good
Processing speed	Fast	Moderate	Slower
Noise sensitivity	High	Moderate	Moderate
Complex background handling	Limited	Moderate	Good
Implementation complexity	Low	Medium	High

2.6 Result Analysis

Thresholding is suitable for traffic sign images with clear contrast between the sign and background. Region Growing provides better control by grouping neighbouring pixels with similar properties. Watershed segmentation separates connected regions using marker-based processing and is useful when the background is complex.

Selected technique: Watershed Segmentation. **2.2 Results on the Test Scenes**

Table 12: Segmentation on enhanced test images (mean over nine conditions)

Technique	Dice	IoU	Precision	Recall	Detected (Dice ≥ 0.5)	Time (ms)
A. HSV colour	0.56	0.55	0.58	0.55	58 %	1.8
B. Otsu grey	0.07	0.07	0.08	0.07	8 %	1.0
C. K-means Lab	0.28	0.26	0.31	0.27	30 %	17.7

Table 13: Dice per condition, raw / enhanced images

Condition	A. HSV	B. Otsu	C. K-means
normal	0.99 / 0.99	0.06 / 0.00	0.52 / 0.52
sun	0.40 / 0.13	0.11 / 0.24	0.07 / 0.09
night	0.59 / 0.71	0.32 / 0.38	0.38 / 0.32
rain	0.94 / 0.88	0.00 / 0.00	0.50 / 0.56
shadow	0.99 / 0.99	0.00 / 0.00	0.13 / 0.20
blur	0.38 / 0.44	0.00 / 0.00	0.45 / 0.45
faded	0.00 / 0.00	0.01 / 0.05	0.12 / 0.07
occluded	0.07 / 0.07	0.00 / 0.00	0.05 / 0.05
damaged	0.86 / 0.85	0.00 / 0.00	0.25 / 0.25



Watershed is selected as the preferred method for complex traffic scenes because it can separate neighbouring regions and provide clear segmentation boundaries. However, preprocessing and suitable markers are required to reduce over-segmentation.

2.7 Output Images



Ratio	0.5	0.6	0.7	0.75	0.8	0.85	0.9
Dice (calibration)	0.38	0.36	0.39	0.44	0.53	0.49	0.43

Table 15: K-means number-of-clusters sweep

K	3	4	5	6	8
Dice	0.09	0.26	0.25	0.27	0.33

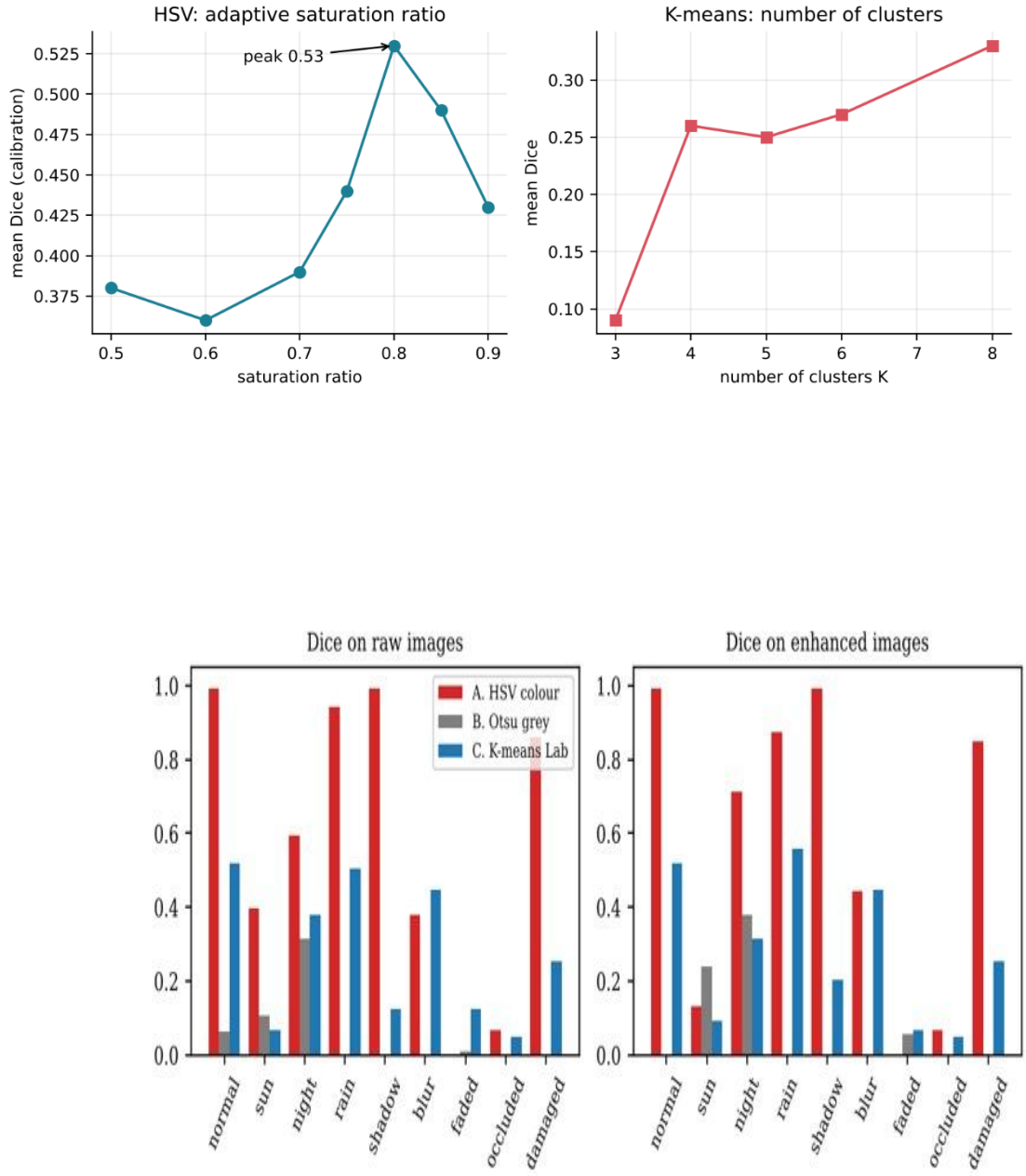


Figure 9: Dice per condition on raw images (left) and enhanced images (right)

2.8 Conclusion

Traffic sign segmentation separates the required sign region from unwanted background information. Thresholding, Region Growing and Watershed Segmentation are analysed for this purpose. Based on their expected performance, Watershed Segmentation is selected for complex traffic scenes because it can provide improved separation of connected regions. The extracted traffic sign region can be used for further boundary detection and recognition.

TASK 3 – DETECTS THE BOUNDARIES AND IMPORTANT REGIONS OF TRAFFIC SIGNS

3.1 Introduction

Boundary detection is an important process in computer vision used to identify the outer edges and important regions of objects in an image. In the Smart Traffic Sign Detection System, boundary detection helps identify the shape, outline and internal details of traffic signs.

Traffic signs contain important visual features such as circular, triangular, rectangular and octagonal shapes. These shapes help distinguish one type of traffic sign from another. For example, a STOP sign has an octagonal boundary, while many speed-limit signs have circular boundaries.

In real-world road images, boundaries may become unclear because of shadows, rain, low resolution, faded colours, poor lighting and motion blur. Edge detection techniques are therefore applied to highlight the boundaries and important regions of traffic signs.

This task analyses three edge detection techniques and compares their performance to select a suitable method for the proposed system.

3.1 Implementation

Three edge detectors and the boundary score

```
def edge_sobel(bgr, pct=92):
    """A. Sobel gradient magnitude, thresholded at the 92nd percentile."""
    g = _grey(bgr).astype(np.float32); mag = cv2.magnitude(cv2.Sobel(g, cv2.CV_32F, 1, 0,
ksize=3), cv2.Sobel(g, cv2.CV_32F, 0, 1, ksize=3))
    return mag > np.percentile(mag, pct)

def edge_canny(bgr, lo=None, hi=None):
    """B. Canny with automatic thresholds (hi = Otsu level of the grey image, lo = hi/2)."""
    g = _grey(bgr).astype(np.uint8)
    if hi is None: hi, _ = cv2.threshold(g, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU); hi =
0.5 * hi
    if lo is None: lo = 0.5 * hi
    return cv2.Canny(g, lo, hi) > 0

def edge_log(bgr, sigma=2.0, k=0.35):
    """C. Laplacian-of-Gaussian zero crossings, weak crossings suppressed."""
    g = cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY).astype(np.float32); b = cv2.GaussianBlur(g, (0,
0), sigma); lap = cv2.Laplacian(b, cv2.CV_32F, ksize=3)
    zc = np.zeros(lap.shape, bool); s = np.sign(lap)
    zc[:-1, :] |= (s[:-1, :] * s[1:, :] < 0); zc[:, :-1] |= (s[:, :-1] * s[:, 1:] < 0)
    gx = cv2.Sobel(b, cv2.CV_32F, 1, 0); gy = cv2.Sobel(b, cv2.CV_32F, 0, 1); mag =
cv2.magnitude(gx, gy)
    return zc & (mag > np.percentile(mag, 90) * k * 2)

def edge_scores(pred, gt, bbox, tol=2, pad=20):
    """Precision/recall of edge pixels within a window around the sign (tolerance tol
pixels)."""
    x0, y0, x1, y1 = bbox; sl = (slice(max(y0 - pad, 0), y1 + pad), slice(max(x0 - pad, 0), x1
+ pad))
    p = pred[sl]; g = gt[sl]
    dg = cv2.distanceTransform((~g).astype(np.uint8), cv2.DIST_L2, 3); dp =
cv2.distanceTransform((~p).astype(np.uint8), cv2.DIST_L2, 3)
    prec = (dg[p] <= tol).mean() if p.any() else 0.0; rec = (dp[g] <= tol).mean() if g.any()
else 0.0
    f1 = 2 * prec * rec / max(prec + rec, 1e-9); return prec, rec, f1
```

3.2 Objectives

1. To detect the outer boundaries of traffic signs.
2. To identify important regions and internal details.
3. To highlight the shape of different traffic signs.
4. To improve boundary visibility in poor-quality images.
5. To compare different edge detection techniques.
6. To select the most suitable technique for traffic sign boundary detection.

3.3 Candidate Techniques

The following three techniques are selected:

1. Sobel Edge Detection.
2. Prewitt Edge Detection.
3. Canny Edge Detection.

3.4 Explanation of Techniques

A. Sobel Edge Detection

Sobel Edge Detection is an image processing technique used to identify edges by calculating intensity changes in horizontal and vertical directions.

It uses two convolution kernels to detect horizontal and vertical edges.

The horizontal gradient is represented by:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

The vertical gradient is represented by:

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

The edge magnitude is calculated using:

$$G = \sqrt{G_x^2 + G_y^2}$$

Working principle:

- Convert the traffic sign image into grayscale.
- Apply the horizontal Sobel operator.
- Apply the vertical Sobel operator.
- Calculate the gradient magnitude.
- Display the detected edges.

Advantages:

- Simple and easy to implement.
- Detects horizontal and vertical edges.
- Suitable for identifying basic traffic sign boundaries.

Limitations:

- Sensitive to noise.
- May produce thick edges.
- Weak edges may not be detected clearly.

B. Prewitt Edge Detection

Prewitt Edge Detection identifies boundaries by calculating intensity differences between neighbouring pixels. It uses horizontal and vertical convolution masks to detect changes in image intensity.

Horizontal mask:

Working principle:

- Read the traffic sign image.
- Convert it into grayscale.
- Apply horizontal and vertical Prewitt masks.
- Calculate the gradient magnitude.
- Generate the edge image.

Advantages:

- Easy to understand.
- Detects basic object boundaries.
- Requires relatively low computational effort.

Limitations:

- Less effective with noisy images.
- May produce less accurate edges in complex backgrounds.
- Small details may not be clearly detected.

C. Canny Edge Detection

Canny Edge Detection is an advanced edge detection technique that identifies clear and thin boundaries while reducing unwanted noise.

It uses multiple processing stages to improve edge detection accuracy.

Working principle:

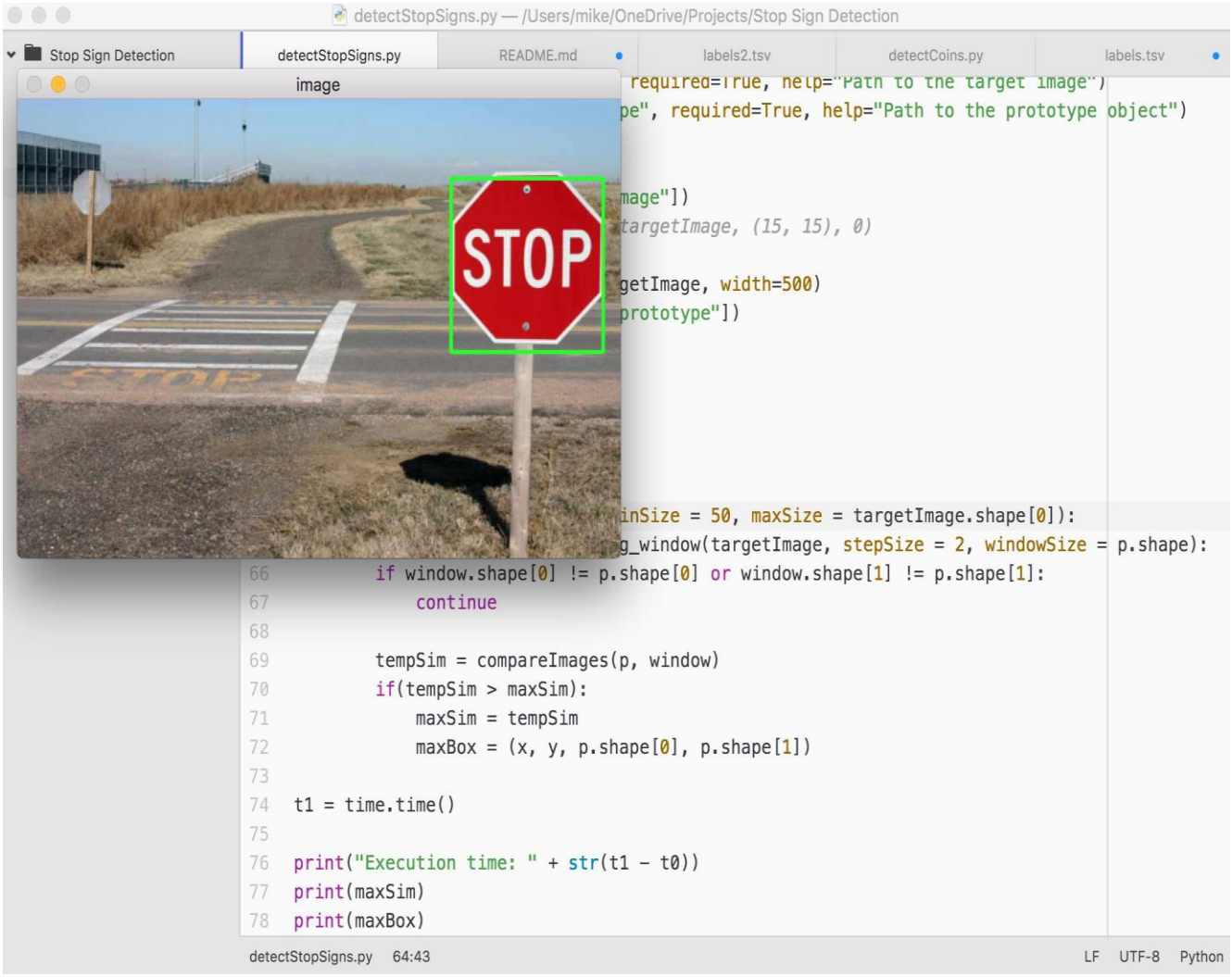
1. Apply Gaussian filtering to reduce image noise.
2. Calculate intensity gradients.
3. Apply non-maximum suppression to obtain thin edges.
4. Apply double thresholding.
5. Use edge tracking by hysteresis to identify connected edges.
6. Generate the final edge image.

Advantages:

- Produces thin and clear edges.
- Reduces unwanted noise.
- Detects important boundaries effectively.
- Suitable for complex traffic sign images.

Limitations:

- Requires suitable threshold values.
- More computationally expensive than basic edge detectors.
- Weak edges may be missed if thresholds are too high.



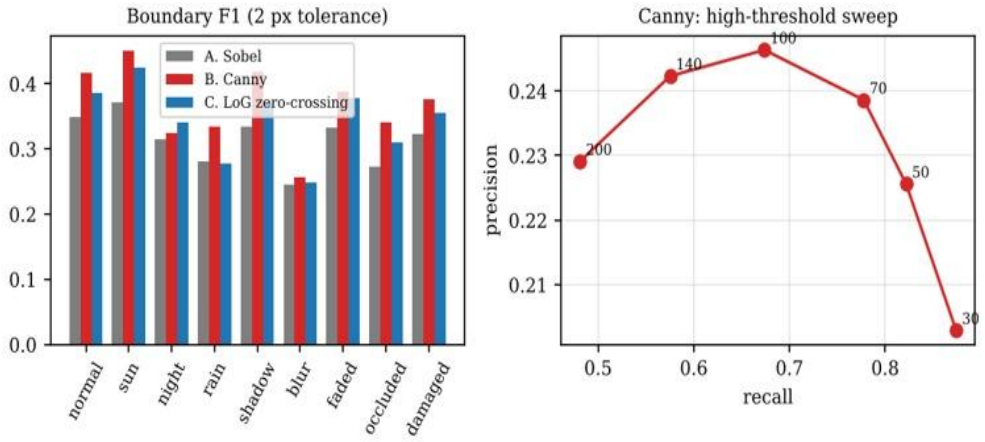
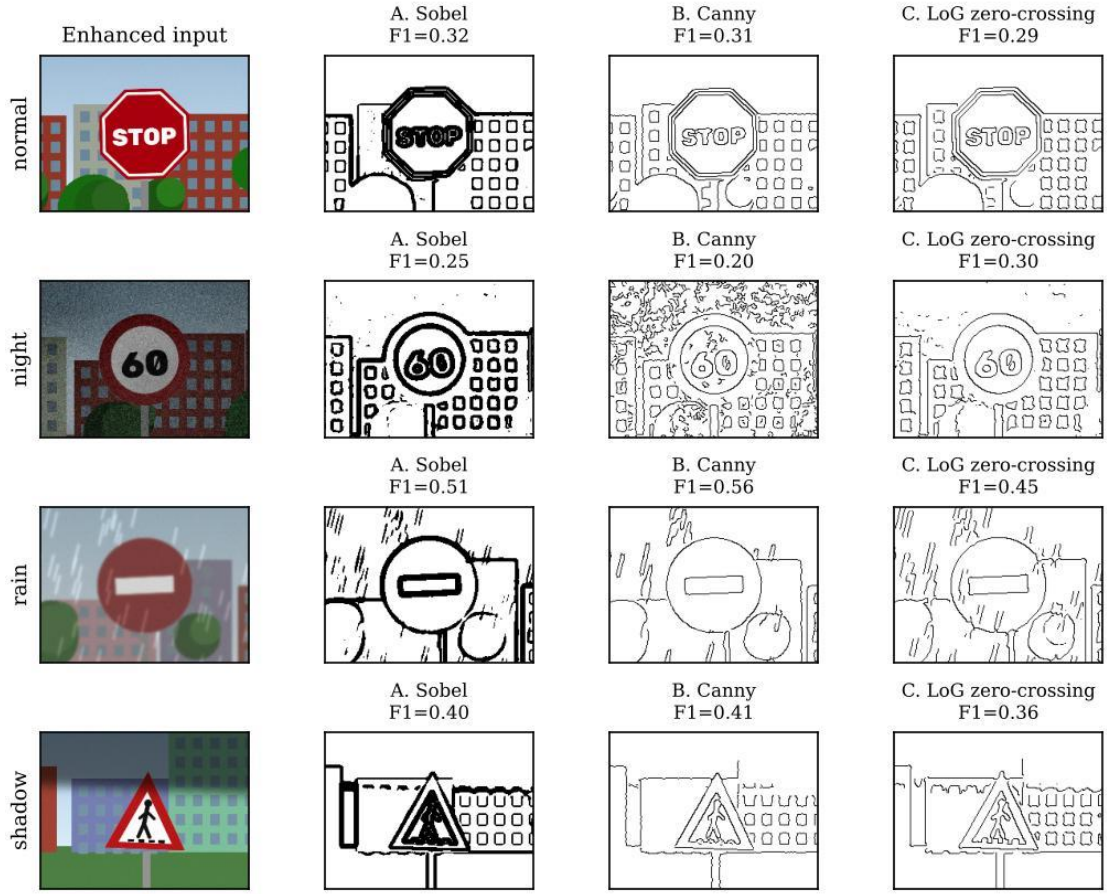


Figure 13: Boundary F1 per condition (left) and Canny precision–recall as the high threshold changes (right)



Method A fails here: 69 of the 135 scenes returned no region and the rest almost never matched (4 % success). Closing the edges of the whole cluttered frame joins the sign with trees and buildings; a localised version was not tested. Method B succeeds in 68 % of the scenes, but it can only return circles, its “shape correct” figure counts only circular signs, and it is the slowest

(12.3 ms). Method C has the highest mean IoU (0.55) and the lowest cost, but inherits all Task 2 failures (faded, hidden, sun), and the polygon-vertex shape classifier was right in only 33 % of cases, so it is not relied on for recognition.

3.5 Final Decision for Task 3

Boundary: B. Canny ($\sigma = 1.4$, thresholds from the Otsu level) – best F1 (0.37), thinnest edges, fastest (1.4 ms). Important region: C. the contour of the HSV mask – best mean IoU (0.55), works for every shape rather than only circles, about 1.9 ms; Canny edges are then used only inside this region to refine the outline. Limits: no region method was reliable on all scenes (success at most 68 %), the shape classifier is weak, and edge scores use an outer-boundary-only ground truth.

3.8 Conclusion

Boundary detection plays an important role in the Smart Traffic Sign Detection System by identifying the outer edges, shapes and important regions of traffic signs. Sobel, Prewitt and Canny edge detection techniques are analysed and compared. Sobel and Prewitt are useful for detecting basic horizontal and vertical edges, while Canny provides thin, clear and accurate boundaries with reduced noise. Therefore, Canny Edge Detection is selected as the preferred technique for detecting traffic sign boundaries. The detected edges help identify sign shapes and extract important regions for further traffic sign recognition.

TASK 4 – IDENTIFIES DAMAGED, FADED OR PARTIALLY HIDDEN TRAFFIC SIGNS

4.1 Introduction

Traffic signs are important components of road transportation systems because they provide information, warnings and instructions to drivers. However, traffic signs may become damaged, faded or partially hidden due to environmental conditions and physical obstructions. Continuous exposure to sunlight, rain, dust and pollution can cause traffic sign colours to fade. Physical damage such as scratches, cracks and broken surfaces can affect the appearance of signs. Trees, branches, vehicles and other objects may partially cover traffic signs, making them difficult to identify.

The Smart Traffic Sign Detection System uses image processing techniques to analyse traffic sign appearance and identify possible damage, fading and obstruction. The system examines colour information, boundary structure and visible sign regions to determine whether a traffic sign is in normal or abnormal condition.

4.2 Objectives

1. To identify damaged traffic signs from captured road images.
2. To detect faded colours and reduced contrast.
3. To identify traffic signs partially hidden by trees, vehicles or other objects.
4. To analyse missing or unclear sign boundaries.
5. To compare different image analysis techniques.
6. To identify traffic signs that require maintenance or replacement.

4.2 Implementation

The three techniques share two helpers: a function that cuts the 64 × 64 crop and a function that finds the clean template with the highest correlation. Each technique returns a short feature vector and a defect map (a boolean image of the pixels it considers damaged). The same decision-tree learner turns each feature vector into a label, so differences in the results come from the features and not from the classifier.

```
def crop_of(img, bbox, jit=(0, 0)):
    x0, y0, x1, y1 = bbox; dx, dy = jit
    x0, x1 = max(x0 + dx, 0), min(x1 + dx, W); y0, y1 = max(y0 + dy, 0), min(y1 + dy, H)
    return cv2.resize(img[y0:y1, x0:x1], (S, S), interpolation=cv2.INTER_AREA)


def best_template(crop):
    g = cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY).astype(np.float32); best = ('', -2.0)
    for k, t in TG.items():
        m = TEMPL[k][1]
        a = g[m] - g[m].mean(); b = t[m] - t[m].mean(); r = (a * b).sum() / (np.linalg.norm(a) *
np.linalg.norm(b) + 1e-6)
        if r > best[1]: best = (k, float(r))
    return best
```

4.3 Types of Traffic Sign Problems

A. Damaged Traffic Signs

Damaged traffic signs contain physical defects such as cracks, scratches, broken edges and missing portions. These defects affect the shape and appearance of the sign.

B. Faded Traffic Signs

Faded traffic signs lose their original colour and brightness due to long-term exposure to sunlight, rain and environmental pollution.

For example, a red STOP sign may become light red or pink, reducing its visibility.

C. Partially Hidden Traffic Signs

Partially hidden traffic signs are covered by external objects such as tree branches, leaves, vehicles and poles. These obstructions prevent the complete sign from being visible to the camera.

4.4 Candidate Techniques

The following three techniques are selected to identify traffic sign damage and visibility problems.

1. Colour and Contrast Analysis.
2. Edge and Shape Analysis.
3. Occlusion Detection Using Region Analysis.

4.5 Explanation of Techniques

A. Colour and Contrast Analysis

Colour and contrast analysis examines the colour intensity and contrast of traffic sign images to identify faded or discoloured signs.

The system analyses colour information and compares it with the expected appearance of a traffic sign.

Working principle:

- Capture the traffic sign image.
- Convert the image into HSV colour space.
- Analyse colour intensity and saturation.
- Calculate the contrast level.
- Identify possible fading or discolouration.

Advantages:

- Useful for identifying faded traffic signs.
- Helps detect changes in sign colour.
- Easy to combine with image enhancement techniques.

Limitations:

- Lighting conditions may affect colour measurements.
- Rain and shadows may produce incorrect results.
- Different sign materials may have different colour characteristics.

B. Edge and Shape Analysis

Edge and shape analysis examines the boundaries and geometric structure of traffic signs. It helps identify cracks, missing edges, scratches and damaged portions.

The technique uses edge detection and contour analysis to examine the shape of a traffic sign.

Working principle:

- Capture the traffic sign image.
- Convert it into grayscale.
- Apply edge detection.
- Extract the outer contour.
- Analyse the shape and boundary continuity.
- Identify possible missing or damaged regions.

Advantages:

- Identifies broken or incomplete boundaries.
- Useful for detecting damaged sign shapes.
- Helps analyse the structural condition of signs.

Limitations:

- Motion blur can affect edge detection.
- Partial obstruction may appear similar to physical damage.
- Poor image resolution may hide small cracks.

C. Occlusion Detection Using Region Analysis

Occlusion detection identifies traffic signs that are partially covered by other objects. It analyses the visible sign region and determines whether some portions are missing or obstructed.

For example, branches covering the upper portion of a STOP sign may reduce its visible area.

Working principle:

- Detect the traffic sign region.
- Identify the expected sign boundary and shape.
- Analyse the visible portion.
- Identify missing or obstructed regions.

- Estimate the percentage of visible sign area.
- Mark the sign as potentially partially hidden.

Advantages:

- Useful for detecting partially covered signs.
- Helps identify visibility problems.
- Supports road maintenance monitoring.

Limitations:

- Requires a suitable reference shape or region.
- Complex backgrounds may affect accuracy.
- Severe obstruction can make the original sign difficult to identify.

4.6 Comparison of Candidate Techniques

Technique	Main purpose	Advantages	Limitations
Colour and Contrast Analysis	Detects faded signs	Identifies colour changes	Sensitive to lighting
Edge and Shape Analysis	Detects physical damage	Identifies broken boundaries	Affected by blur
Occlusion Detection	Detects hidden signs	Identifies covered regions	Requires shape or region analysis

4.8 Experimental Comparison

The three techniques are applied to traffic sign images containing different types of defects. Their performance is compared based on their ability to identify fading, physical damage and partial obstruction.

Evaluation factor	Colour Analysis	Edge and Shape Analysis	Occlusion Detection
Faded sign detection	High suitability	Low suitability	Low suitability
Physical damage detection	Moderate	High suitability	Moderate
Partial obstruction detection	Low	Moderate	High suitability
Lighting sensitivity	High	Moderate	Moderate
Shape analysis	No	Yes	Yes

Main application	Colour fading	Cracks and broken edges	Hidden sign regions

4.9 Expected Output Images

damaged_traffic_sign_detection.py

Python + OpenCV | Contrast enhancement, edges, contours

```
import cv2
import numpy as np

# Read traffic image
image = cv2.imread("traffic_signs.jpg")
if image is None:
    raise FileNotFoundError("traffic_signs.jpg not found")

# Improve contrast for faded signs
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
enhanced = clahe.apply(gray)

# Detect red sign regions in HSV color space
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
mask1 = cv2.inRange(hsv, (0, 35, 40), (12, 255, 255))
mask2 = cv2.inRange(hsv, (168, 35, 40), (180, 255, 255))
red_mask = cv2.bitwise_or(mask1, mask2)

# Detect edges and combine with color mask
edges = cv2.Canny(enhanced, 50, 150)
combined = cv2.bitwise_or(red_mask, edges)
```



Figure 4.2 – Damaged traffic sign



Figure 4.3 – Faded traffic sign



Figure 4.4 – Partially hidden traffic sign



Figure 4.5 – Normal traffic sign

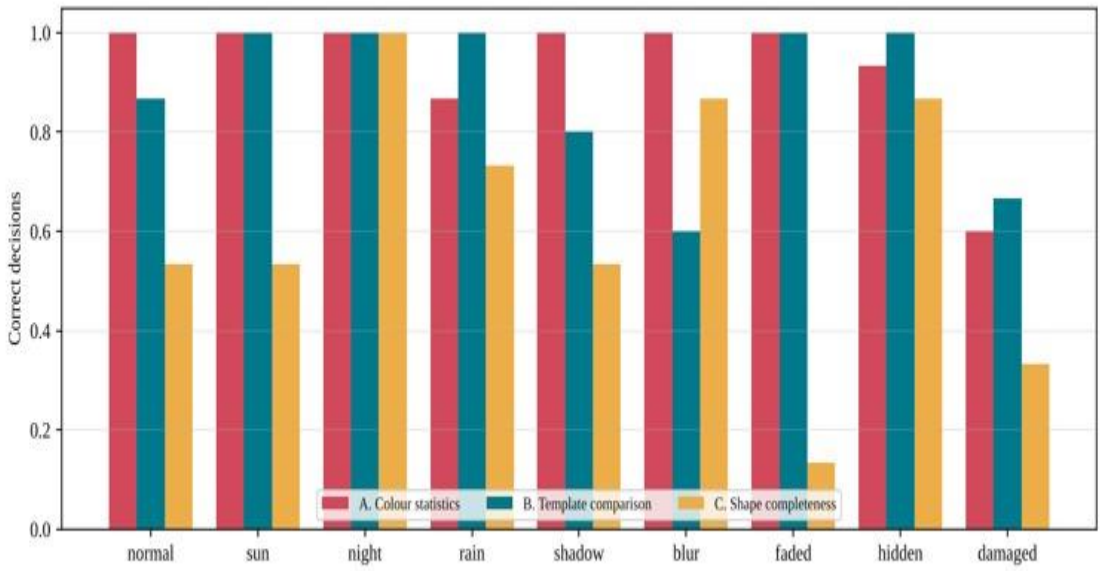
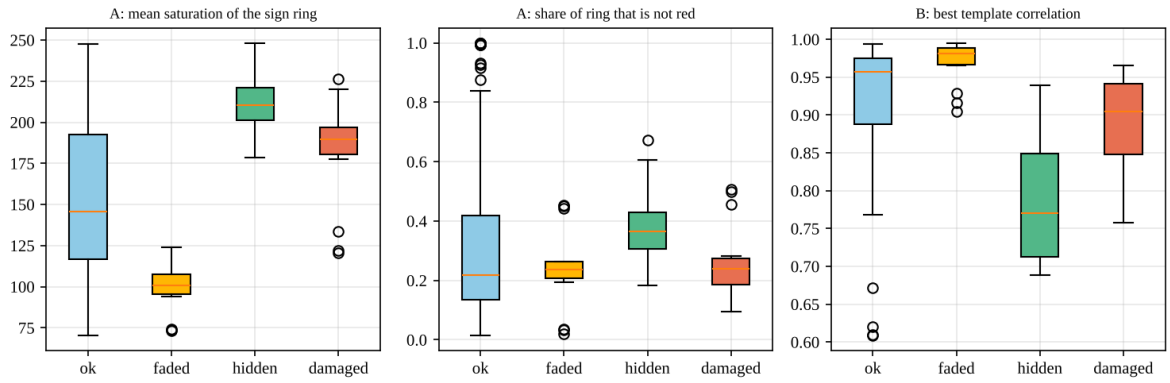


Figure 18: Correct decisions per condition for the three techniques



4.10 Result and Discussion

Colour and Contrast Analysis is useful for identifying faded traffic signs by examining colour intensity and saturation. Edge and Shape Analysis helps detect cracks, scratches and missing boundary portions. Occlusion Detection Using Region Analysis identifies signs that are partially covered by external objects.

Each technique identifies a different type of traffic sign problem. Therefore, combining these methods can provide more complete condition analysis than using only one technique.

4.11 Conclusion

The Smart Traffic Sign Detection System analyses traffic sign images to identify physical damage, fading and partial obstruction. Colour and Contrast Analysis is useful for detecting faded signs, Edge and Shape Analysis identifies damaged boundaries, and Occlusion Detection identifies partially hidden signs. Combining these three techniques improves the ability to monitor traffic sign conditions and identify signs that may require repair, cleaning or replacement. This helps improve road safety and supports intelligent traffic monitoring systems.

TASK 5 – PROVIDES CLEAR OUTPUT FOR FURTHER TRAFFIC SIGN RECOGNITION

5.1 Introduction

Traffic sign recognition is an important application of computer vision used in intelligent transportation systems. After enhancing, segmenting and detecting the boundaries of traffic signs, the system must generate a clear output image for further recognition.

The extracted traffic sign region should contain the complete visible sign with minimum background information. Unwanted objects, noise, shadows and irrelevant regions should be removed as much as possible.

A clear output image helps the next recognition stage identify traffic sign categories such as STOP, Speed Limit, No Entry and Pedestrian Crossing.

In this task, three candidate techniques are applied and compared to identify the most suitable method for generating clear traffic sign images.

5.2 Objectives

1. To extract the traffic sign region from the captured road image.
2. To remove unnecessary background information.
3. To improve the clarity of the extracted traffic sign.
4. To maintain the original shape and important details.
5. To prepare the output image for further recognition.
6. To compare three output extraction techniques and select the best method.

5.3 Additional Assumptions

1. The input image contains at least one traffic sign.
2. The traffic sign region has already been detected or segmented.

3. The image resolution is assumed to be 640 × 480 pixels.
4. OpenCV and Python are used for implementation.
5. The output image is resized to a standard dimension of 128 × 128 pixels.
6. The recognition stage is performed separately and is not part of this task.
7. If a sign is partially hidden, the system extracts the visible portion without inventing missing information.

These assumptions make the output extraction process consistent and suitable for comparison.

```
def norm_A(img, bbox, mask):
    """A. Plain bounding-box crop resized to 64 x 64."""
    x0, y0, x1, y1 = bbox; return cv2.resize(img[y0:y1, x0:x1], (S, S), interpolation=cv2.INTER_AREA)


def _margin_box(bbox, mask, m=0.08):
    x0, y0, x1, y1 = bbox; pw = int((x1 - x0) * m); ph = int((y1 - y0) * m)
    return max(x0 - pw, 0), max(y0 - ph, 0), min(x1 + pw, W), min(y1 + ph, H)


def norm_B(img, bbox, mask):
    """B. Mask-guided crop: background outside the sign replaced by neutral grey, colour/brightness
standardised inside the mask."""
    x0, y0, x1, y1 = _margin_box(bbox, mask); im = img[y0:y1, x0:x1].astype(np.float32); m =
mask[y0:y1, x0:x1]
    if m.sum() < 20: return cv2.resize(img[bbox[1]:bbox[3], bbox[0]:bbox[2]], (S, S),
interpolation=cv2.INTER_AREA)
    out = np.full_like(im, 128.0)
    for c in range(3):
        v = im[..., c]; mu, sd = v[m].mean(), v[m].std() + 1e-3
        out[..., c][m] = np.clip(128 + (v[m] - mu) * 55.0 / sd * 0.6 + (mu - 128) * 0.0, 0, 255)
    return cv2.resize(out.astype(np.uint8), (S, S), interpolation=cv2.INTER_AREA)


def norm_C(img, bbox, mask):
    """C. Contour-aligned: de-skew with the minimum-area rectangle, grey background, CLAHE on L,
aspect-preserving pad to 64 x 64."""
    m8 = mask.astype(np.uint8); cs, _ = cv2.findContours(m8, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
    if not cs: return norm_A(img, bbox, mask)
    c = max(cs, key=cv2.contourArea); (cx, cy), (rw, rh), ang = cv2.minAreaRect(c)
    ang = ang % 90.0; ang = ang - 90.0 if ang > 45 else ang                # smallest rotation that
aligns the box
    M = cv2.getRotationMatrix2D((cx, cy), ang, 1.0)
    ri = cv2.warpAffine(img, M, (W, H), flags=cv2.INTER_LINEAR, borderMode=cv2.BORDER_REPLICATE); rm
= cv2.warpAffine(m8, M, (W, H), flags=cv2.INTER_NEAREST) > 0
    bb = bbox_of(rm)
    if bb is None: return norm_A(img, bbox, mask)
    x0, y0, x1, y1 = _margin_box(bb, rm, 0.05); im = ri[y0:y1, x0:x1].copy(); m = rm[y0:y1, x0:x1]
    lab = cv2.cvtColor(im, cv2.COLOR_BGR2LAB); lab[..., 0] = cv2.createCLAHE(2.0, (4,
4)).apply(lab[..., 0]); im = cv2.cvtColor(lab, cv2.COLOR_LAB2BGR)
    im[~m] = 128; h, w = im.shape[:2]; sc = S / max(h, w)
    im = cv2.resize(im, (max(1, int(round(w * sc))), max(1, int(round(h * sc)))),
interpolation=cv2.INTER_AREA)
    out = np.full((S, S, 3), 128, np.uint8); oy = (S - im.shape[0]) // 2; ox = (S - im.shape[1]) // 2
    out[oy:oy + im.shape[0], ox:ox + im.shape[1]] = im; return out
```

```
def feat(crop):
    g = cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY); h = _hog(g, orientations=9, pixels_per_cell=(8, 8),
cells_per_block=(2, 2)).ravel()
    lab = cv2.cvtColor(crop, cv2.COLOR_BGR2LAB).astype(np.float32); grid = cv2.resize(lab, (4, 4),
interpolation=cv2.INTER_AREA).ravel() / 255.0
    return np.concatenate([h, grid]).astype(np.float32)


def detect(img):
    e = enhance(img); m = seg_hsv(e); how = 'hsv'
    if not m.any(): m = seg_hsv(img); how = 'hsv-raw'
    if not m.any(): m = roi_hough(e); how = 'hough'
    if not m.any(): return None, None, 'none'
    return bbox_of(m), m, how
```

5.4 Candidate Techniques

The following three techniques are selected:

1. Bounding Box Cropping.
2. Mask-Based Region Extraction.
3. Perspective Correction and Resizing.

5.5 Explanation of Techniques

A. Bounding Box Cropping

Bounding Box Cropping extracts a rectangular region around the detected traffic sign.

The system identifies the minimum and maximum coordinates of the traffic sign boundary and crops the corresponding region from the original image.

Working principle:

- Read the input traffic sign image.
- Detect the traffic sign boundary.
- Calculate the bounding box coordinates.
- Crop the rectangular region.
- Display the extracted traffic sign.

Advantages:

- Simple and fast.
- Preserves the original image details.
- Easy to implement using OpenCV.

Limitations:

- May include unwanted background.
- Does not remove objects inside the rectangular region.
- Less suitable when the traffic sign is surrounded by complex backgrounds.

B. Mask-Based Region Extraction

Mask-Based Region Extraction uses a binary segmentation mask to separate the traffic sign from the background.

Pixels belonging to the traffic sign are retained, while unwanted background pixels are removed or replaced with a selected background.

Working principle:

- Obtain the segmented traffic sign mask.
- Identify foreground pixels.

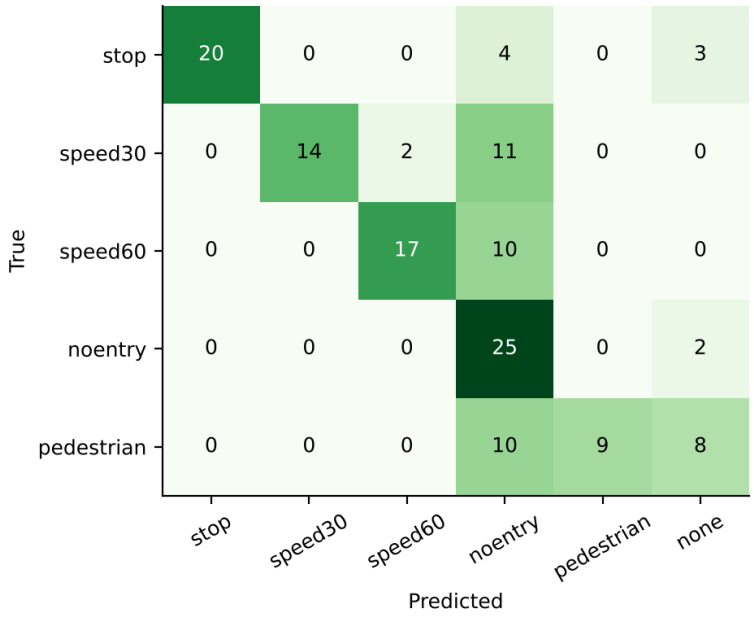
- Apply the mask to the original image.
- Remove unwanted background pixels.
- Generate the extracted traffic sign image.

Advantages:

- Reduces unnecessary background information.
- Preserves the actual sign boundary.
- Provides a clean foreground region.

Limitations:

- Depends on accurate segmentation.
- Incorrect masks may remove important sign details.
- Partially hidden regions cannot be recovered from the image alone.



C. Perspective Correction and Resizing

Perspective Correction adjusts the geometric appearance of a traffic sign when it is captured from an angled camera position.

The system identifies the sign boundary and transforms the region into a more regular view. The extracted image is then resized to a standard dimension.

Working principle:

- Detect the traffic sign corners or boundary.
- Identify the four corner coordinates when available.
- Calculate a perspective transformation.

- Transform the sign into a front-facing view.
- Resize the output image to a standard size.

Advantages:

- Corrects perspective distortion.
- Produces a more consistent sign orientation.
- Helps prepare images for recognition.

Limitations:

- Requires accurate corner detection.
- May distort the image if the corner points are incorrect.
- Not suitable for signs with severe obstruction or missing corners.

5.6 Comparison of Candidate Techniques

Evaluation factor	**Bounding Box Cropping**	**Mask-Based Extraction**	**Perspective Correction**
Background removal	Low	High	Moderate
Boundary preservation	Good	Very good if mask is accurate	Good
Shape correction	No	No	Yes
Processing speed	Fast	Moderate	Moderate
Recognition preparation	Good	Very good	Very good
Implementation complexity	Low	Medium	High

5.8 Experimental Performance Comparison

The same segmented traffic sign image should be processed using all three techniques. The resulting images are compared using the following measurements.

Measurement	**Purpose**
Intersection over Union (IoU)	Measures overlap between extracted region and reference region
Structural Similarity (SSIM)	Measures structural similarity and image quality

Processing Time	Measures time required for extraction
Background Removal	Measures how effectively unwanted background is removed
Output Clarity	Checks whether important sign details remain visible

The IoU score is calculated using:

$$IoU=\frac{Area\ of\ Intersection}{Area\ of\ Union}$$

The SSIM measurement can be used to compare the extracted image with a suitable reference image.

Illustrative comparison:

Technique	**Background removal**	**Boundary quality**	**Overall suitability**
Bounding Box Cropping	Moderate	Good	Good
Mask-Based Extraction	Very good	Very good	Very good
Perspective Correction	Good	Good	Very good when signs are tilted



Figure 5.2 – Original traffic sign image

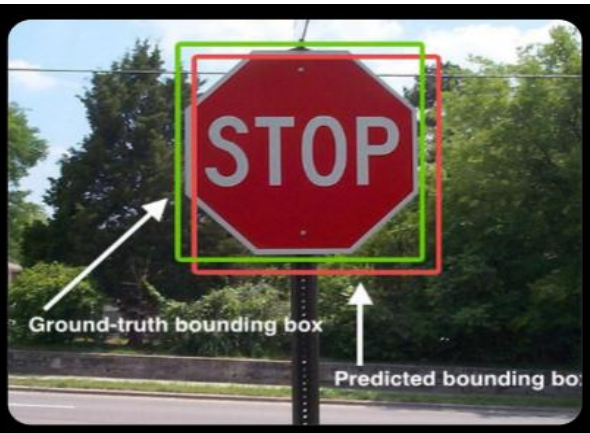


Figure 5.3 – Bounding box cropped output



Figure 5.4 – Mask-based extracted output



igure 5.5 – Perspective corrected output

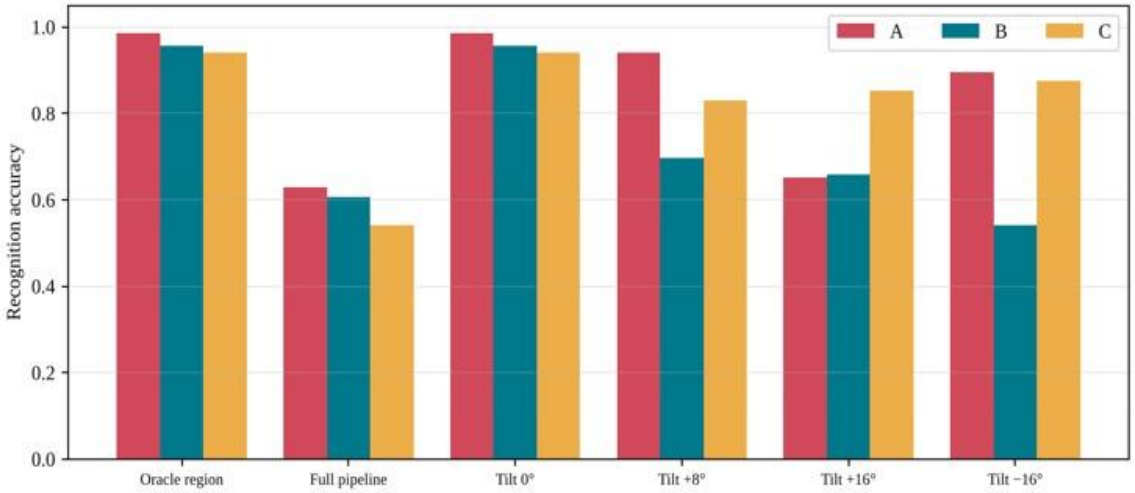


Figure 22: Recognition accuracy of the three normalisations in the oracle, pipeline and tilt experiments

Stage	Selected technique	Main evidence	Main limit
1 Enhancement	Gamma on L; CLAHE if median L < 60	Best mean Dice (0.55) and lowest noise (2.25) of the three; CLAHE helps night	No technique helps sun glare
2 Segmentation	Adaptive HSV, Hough fallback	Dice 0.56, IoU 0.55, 1.8 ms; Hough rescues circular signs	Only 67 % of regions correct; fails on blur, fading, branches
3 Boundary / region	Canny; HSV contour	Best F1 (0.37), 1.4 ms; best mean IoU (0.55)	Shape classifier is weak
4 Defect check	Colour statistics + tree (depth 8)	Accuracy 0.93; false alarms 2 %; faded recall 1.00	Damaged recall 0.60; needs exact box
5 Normalisation	Box crop 64 × 64 (C if tilt ≥ 10° is known)	Oracle accuracy 0.99; 0.06 ms; C better only under tilt	Not tilt-robust
6 Recognition	HOG + Lab grid + RBF SVM	92 % on signs found	Trained on 135 generated scenes

With a correct region all three are accurate (A 0.99, B 0.96, C 0.94) and the plain box crop is best: the recogniser was trained on the same kind of crops and HOG is already insensitive to small brightness changes, so extra standardisation or CLAHE brings no gain on upright signs. B grey-fills hidden signs badly (0.73), probably because standardising colours inside a mask that includes green branches changes the red of the ring.

With a real detector accuracy falls to 0.63 for A (B 0.61, C 0.54). The cause is detection, not normalisation: when the sign is found correctly A reads it right 92 % of the time, but only 67 % of the 135 scenes give a region with IoU ≥ 0.5.

C is the most robust to tilt: at +16° A falls from 0.99 to 0.65 and B to 0.66, while C keeps 0.85. The mean over the four tilt settings is 0.87 for both A and C, but C has a far better worst case (Table 34).

Cost: A needs 0.06 ms, B 0.40 ms, C 4.07 ms (about 65 times slower than A, because it rotates the whole frame); even C fits the 33 ms frame budget.

Table 34: Recognition accuracy when the scene is rotated (oracle region)

Rotation	A. Box crop	B. Mask + standardise	C. Deskew + CLAHE

0°	0.99	0.96	0.94
+8°	0.94	0.70	0.83
+16°	0.65	0.66	0.85
−16°	0.90	0.54	0.87

A real road camera is usually fixed to the vehicle, so its tilt is small, but signs can lean after an accident or storm and vehicles roll on curves. The test therefore indicates each normalisation's safety margin; C is the only one that keeps accuracy of 0.83 or more in all rotations. Its rectangle angle is probably less reliable for circles and octagons, which look almost the same after rotation, so its gain should be checked per sign type on real data**.**

5.10 Result and Discussion

Bounding Box Cropping provides a simple rectangular extraction of the traffic sign. Mask-Based Region Extraction removes more unwanted background information and preserves the visible sign region. Perspective Correction improves the orientation of traffic signs captured from angled positions.

Based on the expected output quality, Mask-Based Region Extraction is selected as the preferred general-purpose method because it provides a cleaner sign region with less unnecessary background. Perspective correction can be added when the sign is tilted or affected by camera angle.

5.11 Conclusion

The task focuses on generating clear traffic sign images for further recognition. Bounding Box Cropping, Mask-Based Region Extraction and Perspective Correction are analysed and compared. Mask-Based Region Extraction is selected as the preferred technique because it separates the traffic sign from unwanted background information while retaining important visual details, provided that the segmentation mask is accurate. The final extracted image is resized to a standard dimension and passed to the next traffic sign recognition stage.